

计算机系统结构实验报告Lab01: Flowing light

uu-matter543

2024-2025-2

摘要

在 Lab01 中，我成功搭建了 Vivado 集成开发环境，学习了如何新建 design 源文件以添加模块、simulation 源文件以添加仿真激励，初步了解了 Verilog 语法、模块设计、仿真方法、调试，为以后的 FPGA 实验打下了坚实基础。

目录

1	实验目的	2
2	原理分析	2
2.1	Vivado 工程文件的基本组成	2
2.2	Flowing light 的原理	2
3	功能实现	3
3.1	激励文件 flowing_light_tb.v 编写	3
3.2	管脚约束 lab01_xdc.xdc 编写	4
4	上板验证	5
5	收获与反思	5

1 实验目的

1. 通过基础实验熟悉 Xilinx 逻辑设计工具 Vivado 开发环境;
2. 了解硬件描述语言 Verilog HDL 描述功能行为的逻辑;
3. 通过仿真检验电路设计是否预期;
4. 学习使用 I/O Planning 添加管脚约束;
5. 实现 Flowing light 的功能;
6. 熟悉系统硬件开发的基本实验流程。

2 原理分析

2.1 Vivado 工程文件的基本组成

- design source (.v文件) (硬件功能设计文件)
- simulation source (.v文件) (仿真输入激励文件)
- constraints (.xdc文件) (上板所需管脚约束文件)

2.2 Flowing light的原理

Flowing light 要求在一段时间内, 8 个 LED 灯依次轮流亮灭, 最后一个 LED 熄灭后, 第一个 LED 循环亮起。这个功能可以使用移位来实现控制, 实验指导书中也给出了相应的实现。我的代码如下:

```
module flowing_light(  
    input clock_p,  
    input clock_n,  
    input reset,  
    output [7:0] led  
);  
    reg [23:0] cnt_reg;  
    reg [7:0] light_reg;  
    IBUFGDS IBUFGDS_inst(  
        .O(CLK_i),  
        .I(clock_p),  
        .IB(clock_n)  
    );  
    always @ (posedge CLK_i) begin  
        if (!reset) cnt_reg <= 0;
```

```
        else cnt_reg <= cnt_reg + 1;
    end
    always @ (posedge CLK_i) begin
        if (!reset) light_reg <= 8'h01;
        else if (cnt_reg == 24'hffffff) begin
            if (light_reg == 8'b10000000) light_reg <= 8'b00000001;
            else light_reg <= light_reg << 1;
        end
    end
    assign led = light_reg;
```

定义端口 clock_p 和 clock_n 用于实现差分时钟，reset 用于复位，cnt_reg 用于存储流水的时间间隔，light_reg 即为 8 位流水 LED 的控制信号。

reset=0 时复位为初始状态，reset=1 时，cnt_reg 在每个时钟上升沿 +1，并在每次到达 24 位全 1 时更新 light_reg 状态，即 LED 每过 2^{24} 个时钟周期进行一次移位。两个 always 模块分别实现流水间隔计数和 LED 信号更新，最后用 assign 连接 led 和 light_reg，保证流水灯控制信号的实时更新。

3 功能实现

本实验比较简单，基于上述的原理容易实现 Flowing light 的功能。在实现流水灯的基本逻辑后，编写 flowing_light_tb.v 激励文件用以仿真测试，编写 lab01_xdc.xdc 管脚约束用以上板验证。

3.1 激励文件 flowing_light_tb.v 编写

```
reg clock_p;
reg clock_n;
reg reset;
wire [7:0] led;
flowing_light example_flow_light(
    .clock_p(clock_p),
    .clock_n(clock_n),
    .reset(reset),
    .led(led)
);
parameter PERIOD = 10;
always #(PERIOD*2) clock_p = !clock_p;
```

```
always #(PERIOD*2) clock_n = !clock_n;
initial begin
    clock_p = 1'b0;
    clock_n = 1'b1;
    reset = 1'b1;
    # (PERIOD*2) reset = 1'b0;
    # (PERIOD*4) reset = 1'b1;
end
```

reg clock_p, clock_n 实现差分时钟, reg reset 保存复位信号, 8 位 wire led 用于更新 LED 状态; flowing_light example_floating_light 接入 flowing_light 流水灯模块。根据差分时钟原理, 初始化两个时钟为相反信号, 并设定相同时钟周期; 之后激活 reset 1 次, 保证系统从初态开始运行。

在运行仿真过程中，我发现由于等待 `cnt_reg` 到达 24 位全 1 的时间比较长，需要仿真较长时间，不便于观察。于是修改设计模块中 `cnt_reg` 为 2 位，并在 `cnt_reg` 到达全 1 时进行 LED 状态更新，即可在仿真图上看到流水灯波形如下：



图 1: 流水灯仿真波形图

3.2 管脚约束 lab01_xdc.xdc 编写

根据实验指导书，采用拖动方式实现管脚约束，这里不再赘述。

4 上板验证

根据实验指导书连接设备，将程序生成二进制流文件，并写入开发板，可以在开发板上观察到流水灯效果；按下 reset 后也能观察到流水灯复位，说明流水灯功能已经实现。

5 收获与反思

我之前并未接触过硬件功能设计、仿真与实现，但在老师、助教以及实验指导书的指导下，我还是顺利完成了本次实验。在本次实验中，我了解了 Vivado 开发环境、工程文件组成，体验了 Verilog 开发的基本流程，并实现了流水灯功能，收获颇丰。感谢老师和助教的指导，也感谢课程组用心制作的实验指导书。